Structural Alignment via Causal-Arc Execution: Comprehension as an Alignment Methodology

George Punter*[1]

[1]Condri , george@condri.app

May 2026

Abstract

Behavioural alignment interventions — system prompts, instruction tuning, and preference shaping from reinforcement learning from human feedback — work briefly and then drift. The drift is not random: the underlying training gradient continues to reward the very behaviours that simple rules try to suppress (turn-level deference, minimum-viable-product defaulting, mid-execution check-ins), and across long unsupervised tasks the gradient re-asserts itself. We argue for an alternative we call *structural alignment*: rather than instructing the agent what to do, we give it a first-principles structural account of *what* the situation is, such that the desired behaviour is the natural consequence of the agent's own reasoning rather than a rule applied on top of it. We instantiate this position with a concrete methodology — *causal-arc execution* — that gives the agent a fifty-token activation pointing at a structural account of what a user mandate is (a unit carrying an outcome, a scope, and a delegation of trust). We evaluate the methodology on a blinded benchmark of eighteen cells: two task scales (short and long), three prompt conditions (naive, "keep going until done," and causal-arc), and two frontier models (Claude Opus 4.7 and Sonnet 4.6). On short tasks, causal-arc ties with the simple imperative (4.5/5 vs. 4.5/5). On a long multi-file task with approximately thirty in-scope decisions, causal-arc wins on design coherence specifically — the dimension the methodology is designed to address — with a mean score of 5.0/5 against 4.5/5 for the simple imperative and 3.5/5 for the naive baseline. Naive cells exhibited concrete coherence breakdowns: error envelopes declared and never wired, inconsistent status codes across endpoints, and documentation diverging from shipped behaviour. We interpret these results as evidence for the alignment claim in miniature: a structural activation maintained coherence across long unsupervised execution where a surface imperative could not. We discuss limitations, including small N per cell, and outline future work.

1 Introduction

The dominant paradigm for aligning large language model agents with user intent is *behavioural*: the agent is instructed, via a system prompt, a fine-tuned policy, or a preference model derived from human feedback, to produce certain kinds of outputs and avoid others [Ouyang et al., 2022, Bai et al., 2022, Christiano et al., 2017]. The intervention sits at the level of behaviour — "produce outputs like this, not like that." This approach is broadly effective for the kinds of short, well-bounded tasks on which it is typically trained and evaluated.

It is also known to degrade under load. As agents are deployed on longer, more autonomous tasks — multi-file software builds, long refactors, research arcs that span many decisions without intervening user

*MEng Electronic and Information Engineering (1st Class Honours), Imperial College London.

feedback — behaviours that the surface alignment was supposed to suppress re-emerge [Perez et al., 2022, Sharma et al., 2024, Kenton et al., 2021]. The agent mid-execution stops to ask whether to continue. The agent produces a minimal version of the requested artifact and asks whether to expand. The agent flags routine in-scope steps as "potentially impactful" and pauses for confirmation. The agent declares an error-handling contract in code or documentation and never implements it. These are not random failures; they are a predictable consequence of how instruction-tuned models are shaped by turn-level reward signals, which systematically reward cautious-looking turns at the expense of mandate-level execution [Ji et al., 2023, Leike et al., 2018].

We frame this as a tension between two kinds of alignment. *Behavioural alignment* sits on top of the model's existing gradient as a rule; under load the gradient wins. *Structural alignment* replaces the model's representation of the situation: if the agent has integrated a first-principles account of what a user request structurally *is*, the cautious-looking move (fragmenting the mandate) is no longer cautious in the agent's own reasoning, and the discipline holds because the reasoning itself now points the right way. Suppressing an equilibrium is hard. Replacing the model of the situation that produces the equilibrium is, we argue, easier and more durable.

We make this concrete by presenting a single methodology in detail and benchmarking it against two natural comparison conditions. The methodology, *causal-arc execution*, gives the agent a structural account of what a mandate is (a unit carrying an outcome, a scope, and a delegation of trust) and what fragmenting that unit costs (degraded outcome, shrunk scope, returned trust). The activation is roughly fifty tokens of prompt; the substance is the first-principles content the activation points at. We hypothesise that, where short tasks can be carried by sheer momentum, *long unsupervised tasks require a frame the agent can use to maintain coherence across many in-scope decisions* — and that a structural frame will outperform a behavioural imperative on precisely this dimension.

We test this hypothesis on a blinded benchmark of eighteen cells. The result, summarised in the abstract, is that the methodology ties with a strong behavioural imperative on short tasks and wins on long-task design coherence: the dimension on which it was predicted to win. We do not claim this single benchmark is conclusive. We do claim it is evidence consistent with the structural-alignment framing, and inconsistent with a story in which causal-arc is just a verbose rephrasing of "keep going."

The contributions of this paper are: (1) a structural argument distinguishing behavioural and structural alignment, with a concrete test that discriminates them; (2) a fully specified methodology, causal-arc execution, derived from that argument; (3) a blinded eighteen-cell benchmark across two task scales, three prompt conditions, and two frontier models; and (4) a discussion of the conditions under which structural framing pays off and the conditions under which behavioural imperatives suffice.

2 The Methodology

2.1 What a mandate is, structurally

A user request to an AI agent is not a string. It is a compressed unit of intention that carries three things simultaneously.

A desired outcome. Something the user wants to exist by the end of the interaction. A working CLI tool. A clean refactor. A draft of a paper. The outcome is the thing the mandate is *about*.

An implicit scope. A region of decision-making the user has pre-authorised. When a user says "build a TODO CLI with add, list, done, and remove subcommands and JSON persistence," they have implicitly decided that the agent should pick the file structure, the function names, the test layout, the error-handling

style, and many other in-scope details. They have decided, by not asking about those things, that they do not need to decide them.

An implicit trust transaction. The user has spent their own attention deciding to delegate this work. That delegation is a finite resource: it cost something to give, and it expires if not consumed by useful execution.

We call the union of these three the *mandate*. The mandate is the unit, not the words. The words are how the unit gets compressed for transmission. The agent's job is to receive the mandate, identify its three components, and consume it through to a landing.

2.2 Why fragmentation degrades alignment

The dominant failure mode we observe across long-task agent execution is *fragmentation*: the agent interrupts the arc to ask the user about something that was already inside the mandate. "Want me to continue?" "I built the minimal version, want me to add the rest?" "Should I include input validation?" Each of these looks, at the level of a single turn, like a cautious move. At the level of the mandate it is corrosive.

Fragmentation degrades each of the three components of the mandate:

It degrades the outcome. Where an outcome was supposed to exist at the end, what now exists is a *question about the outcome*. The user must spend more attention to convert the question back into the outcome. The deliverable has been replaced by a strictly weaker artifact.

It degrades the scope. The user already decided the scope by not asking about details inside it. When the agent asks back about one of those details, the agent is returning a decision the user had already made — the scope shrinks, and what was one mandate becomes two smaller, partial, recontextualised ones.

It degrades the trust. The delegation was a finite transaction. Fragmentation returns the delegation unused; the user must spend the attention again. Worse, repeated fragmentation teaches the user that delegation to this agent does not actually offload work, which trains the user toward smaller, more surveilled prompts in future — compounding the failure mode.

The structural claim is therefore: *the agent that fragments is not being cautious. It is failing to consume the mandate it received.* This is not a stylistic preference; it is a description of what the mandate *is* and what fragmenting it does to its components.

The training dynamics that produce fragmentation are well documented. Instruction-tuned models are shaped by feedback signals that are largely turn-level: a human rater sees a single response and judges it. From that vantage, a turn that asks a clarifying question looks safer than one that decides; a turn that produces an MVP and asks about extension looks more careful than one that produces the full thing; a turn that pauses before a "potentially impactful" step looks more responsible than one that proceeds. At the turn level, these judgements are reasonable. Across the full task, they are catastrophic, because the turn-level training never saw the longer task [Bai et al., 2022, Sharma et al., 2024].

The training gradient therefore points *toward* fragmentation. A surface rule like "do not ask permission mid-task" is being applied on top of a gradient that pushes the other way. Under load, the gradient wins.

2.3 The causal-arc activation block

The methodology's runtime surface is a short prompt block that points the agent at the structural account above. A representative form is:

Execute this as a single causal arc. Make decisions inside scope; do not pause for permission.

Verify your own work as you go. Report at the landing, not at the steps.

followed by the task description.

The block is roughly fifty tokens. It does not, on its surface, contain the structural argument; it points at it. The argument is loaded by the agent's recognition of what "mandate," "scope," "causal arc," and "landing" refer to. For models trained on extensive natural-language priors, this recognition is reliable; the activation behaves more like a frame invocation than a rule.

A small number of variants emphasise different facets of the same frame ("No MVP — full build," "Decide and proceed where the scope is clear," "Until completion"). They are not separate methodologies; they are different entry points to the same structural account.

2.4 The structural shape: pre-arc, mid-arc, post-arc

Given the account above, the correct shape of execution is a single causal arc with three phases.

Pre-arc reception. The agent reads the mandate and identifies what it is: outcome, scope, and trust transaction. It distinguishes *scope-level ambiguity* (the mandate is unclear about what the user wants) from *detail-level ambiguity* (the mandate is silent on details inside a clear scope). Scope-level ambiguity is surfaced before the arc begins. Detail-level ambiguity is decided inside the arc, by the agent, using its in-scope authorisation.

Mid-arc execution. The agent proceeds through the work. It makes in-scope decisions as they arise. It verifies its own work as it goes — running tests, re-reading what it has written, checking the output against the desired outcome. The verification is to itself, not to the user; the user is not the agent's runtime quality check.

Post-arc landing. The agent finishes and reports. The report includes what was built, the in-scope decisions the agent made (surfaced honestly so the user can review them), and any new questions that *having landed the work* genuinely require user attention. New questions can legitimately appear here; they cannot legitimately appear mid-arc on matters the mandate already covered.

The arc is one motion. Mid-arc check-ins break the motion. Pre-arc clarifications on genuinely unclear scope, and post-arc honest landings, are not breaks — they are part of the shape.

2.5 When not to apply the discipline

The methodology has limits. It should not be applied when:

- The mandate's scope is genuinely unclear. "Help me improve my codebase" is scope-ambiguous; the agent should clarify before the arc begins. Scope-level questions up front are not fragmentation.
- A genuine blocker arises mid-arc — a missing credential, a contradictory requirement, an external system that is down. Blockers are not fragmentation; they are reports that the arc cannot complete as given.
- The agent detects intent-divergence: it realises the mandate as stated would not give the user what they actually want. Surface it. Intent-divergence is a report that the arc as-stated points the wrong way, not a fragmentation of a correctly-stated arc.
- The cost of an irreversible step exceeds the trust transferred. If the agent is about to do something the user could not undo and the mandate did not specifically pre-authorise (deleting external data, sending live

messages, spending money), confirm. The trust transaction in a typical "build me X" mandate covers a great deal, but not unbounded irreversibility.

The test is simple: *would executing past this point honour the mandate, or violate it?* If continuing honours it, continue. If continuing would violate it, surface and pause.

2.6 Anti-patterns

We name several recurring failures to give the methodology a vocabulary for transcript review:

- **The check-in tic** — the agent pauses between sub-steps of a single mandate and asks "want me to continue?". The most common fragmentation.
- **The MVP defence** — the agent produces a minimal version of the requested artifact and asks whether to expand. The check-in tic specialised to artifact production.
- **The permission grub** — the agent flags a routine in-scope step as "potentially impactful" and pauses for confirmation. The check-in tic in a costume of care.
- **The pseudo-honesty** — the agent reports half-completion as if it were a finished deliverable while keeping the unfinished half implicit. The inverse failure of fragmentation: the arc does not land, but the report pretends it has.
- **The clarification stall** — the agent asks clarifying questions whose answers are derivable from the mandate itself, before any execution begins.
- **The defer-to-user finish** — the arc has actually completed, but instead of landing the report cleanly, the agent appends a list of optional next steps that crowds out the landing.

Each anti-pattern is a distinct shape; naming them lets reviewers point to specific failures rather than gesturing at vague over-cautiousness.

3 Benchmark Design

We designed an evaluation that would discriminate between three hypotheses: (a) the causal-arc methodology adds nothing beyond a naive prompt; (b) the methodology is equivalent to a simple behavioural imperative such as "keep going until done"; or (c) the methodology maintains coherence on long tasks in a way the imperative does not.

3.1 Task scales, conditions, and models

The benchmark factorial is two task scales $\times$ three prompt conditions $\times$ two models, yielding eighteen cells.

Short tasks. Two short, well-specified tasks were used: a CLI tool (a TODO manager with four subcommands and JSON persistence) and a short technical paper (approximately 1500 words on a specified topic). These tasks have clear scope and a small number of in-scope decisions; they were chosen as a baseline against which to test the long-task hypothesis.

Long task. A single long task: build a complete URL-shortener web service consisting of a FastAPI backend, SQLite persistence, a minimal HTML frontend, a test suite, a Dockerfile, and a README. The

deliverable spans more than ten files and embeds approximately thirty in-scope decisions (URL schema, identifier strategy, status code conventions, error-response shape, persistence patterns, integration approach, documentation contract). The long task is the regime in which the methodology is predicted to differentiate.

Prompt conditions.

- *Naive.* The task description, with no additional framing.
- *Keep-going.* The task description prefixed with "Keep going until done; do not pause to ask."
- *Causal-arc.* The task description prefixed with the causal-arc activation described in Section 2.

Models. Two frontier models, both anonymised here as Claude Opus 4.7 and Claude Sonnet 4.6, sampled at default temperature with a single attempt per cell.

3.2 Blinded scoring

To avoid judge contamination by knowledge of condition, all outputs were anonymised before scoring. Each cell was stripped of any internal references to its condition or model. A separate judge (a different model instance with no access to the mapping) scored each anonymised deliverable against a fixed rubric. The condition-to-cell mapping was disclosed only after all scores were recorded. The full anonymised judging audit trail is released alongside this paper.

We do not claim this single-judge protocol is the strongest possible evaluation; multi-judge inter-rater agreement is identified in Section 6 as important future work. We do claim that the blinded protocol used here is strong enough to rule out the most obvious sources of judge bias (favouring outputs the judge knew came from a particular condition).

3.3 Rubrics

Short tasks were scored on a single 1–5 scale, capturing overall quality.

The long task was scored on a multi-dimensional rubric with five sub-scales of 1–5 each, summing to 25:

- **Functional completeness** — does the deliverable do what the task asks?
- **Test coverage** — are the components tested, and are the tests meaningful?
- **Integration** — does the frontend talk to the backend correctly, and do the persistence and routing layers cohere?
- **Design coherence** — do the choices across the codebase agree with each other? Do schema, handler, persistence, and documentation describe the same system?
- **Documentation** — is the README accurate, and are the contracts described actually delivered?

The design-coherence sub-score is the dimension the methodology was designed to address. The other four are included partly as a check (if causal-arc helped *everything*, the result would be less informative about the specific mechanism we propose) and partly to ground the total score.

4 Results

4.1 Short tasks: aggregate tie

Across the four short-task cells per condition (two tasks $\times$ two models), the aggregate means were:

Condition	Mean score (1–5)
Naive	4.25
Keep-going	4.50
Causal-arc	4.50

Causal-arc and keep-going are tied; both lift the baseline by 0.25. The short-task regime does not discriminate the methodology from the simple imperative. This is the predicted result: short, well-specified tasks can be carried by momentum, and the structural frame does not have a distinct contribution to make.

The one short-task cell where causal-arc had a clear edge was Sonnet on the CLI task. The naive cell produced a working but underspecified tool (score 3); the keep-going cell produced more tests (17 vs. 12) but used manual argument parsing and fragile monkey-patching (score 3); the causal-arc cell used `argparse`, made the `done` subcommand idempotent, and shipped a cleaner test suite (score 4). This is suggestive but on a single cell.

4.2 Long task: total scores

The long-task totals, out of 25:

Model	Naive	Keep-going	Causal-arc
Opus	22	23	**25**
Sonnet	19	**25**	24
Mean	20.5	24.0	**24.5**

Causal-arc and keep-going both substantially outperform the naive baseline. On aggregate, causal-arc edges keep-going by 0.5, but the between-cell variance is large enough that the total-score difference should not be over-read. The interesting signal is on the design-coherence sub-score.

4.3 Design coherence: the predicted dimension

The design-coherence sub-scores, out of 5:

Model	Naive	Keep-going	Causal-arc
Opus	4	4	**5**
Sonnet	3	5	**5**
Mean	3.5	4.5	**5.0**

Causal-arc is the only condition that achieves perfect coherence in both models. The lead over the naive baseline is +1.5; the lead over keep-going is +0.5. The keep-going condition recovers most of the naive gap, but not all of it: a behavioural imperative is enough to keep the agent producing, but not enough to keep the agent producing *consistently across many decisions*.

4.4 Drift documented in naive cells

The blinded judge identified concrete coherence breakdowns in the naive-prompt long-task cells. We reproduce two illustrative findings.

In the `shortener_naive_sonnet` cell, the agent declared an `ErrorResponse` Pydantic model and annotated routes with `responses={404: {"model": ErrorResponse}}`, but never installed the corresponding exception handler. The actual wire format on a 404 was FastAPI's default `{"detail": ...}`, not the declared envelope. The README claimed "consistent JSON error responses" that the running code did not deliver. The test suite covered for the divergence by asserting only that `"detail" in data`, papering over the inconsistency rather than catching it.

In the `shortener_naive_opus` cell, the agent shipped two different 422 error-envelope shapes depending on the source of the validation error: the `RequestValidationError` handler returned `{code, message, details}`, while other 422 returns used `{code, message}`. The `/shorten` endpoint returned 200 where 201 would be conventional. A stray `shortener.db` artifact was committed to the repository.

Both keep-going cells were tighter but contained minor drift. The `shortener_keepgoing_opus` cell had the same 200-vs-201 status code inconsistency and a README idempotency claim without test coverage; the `shortener_keepgoing_sonnet` cell was clean.

The causal-arc cells were the only condition with 5/5 design coherence in both models. The blinded judge wrote of these cells: "The 24–25 cells read like the same author wrote schema, db, html, and README in one sitting. The 19–22 cells show seams: the schema names say one thing, the handler returns another, or the README documents a contract the code does not quite meet."

4.5 The Sonnet keep-going cell

The Sonnet keep-going cell tied causal-arc on the long task (25/25 vs. 24/25). This is worth marking explicitly: the simple imperative was the best condition for at least one (model, task) pair. We do not interpret this as evidence against the methodology; we interpret it as evidence that the methodology's lead is small enough in aggregate to be flipped by single-cell variance, and that "keep going until done" is a remarkably effective simple imperative for a strong model on a task whose scope is clear. The methodology's claim is not that it dominates the imperative uniformly; it is that it maintains coherence specifically, and the coherence sub-score data is the relevant evidence for that narrower claim.

In aggregate, the cross-model pattern is suggestive: Opus benefits more from the causal-arc frame than from the imperative (25 vs. 23), while Sonnet benefits more from the imperative than from the frame on totals (25 vs. 24) but ties on coherence (5 vs. 5). One reading is that the stronger model has more headroom to use a structural frame; the weaker model leans on momentum. With $N = 1$ per cell, this is speculative.

5 Discussion

5.1 The mechanism: comprehension over compliance

The mechanism we propose is straightforward. A long unsupervised task requires the agent to make many in-scope decisions without intervening user feedback. Each decision is locally underspecified; the agent must choose. To keep those choices consistent with one another across many files and many hours of execution, the agent needs an internal frame against which to check consistency.

A behavioural imperative ("keep going") gives the agent momentum but no frame. Each local decision is made in isolation; consistency across decisions is, at best, an emergent property of the model's stylistic priors. The result is the kind of drift documented in the naive and keep-going cells: declared envelopes that don't get wired, status codes that vary across endpoints, documentation that diverges from code.

A structural frame (causal-arc) gives the agent an account of what the mandate is, and therefore what consistency *means* in this context: honouring one unit of intention across many decisions. Coherence becomes a value the agent actively maintains, not a side effect of momentum. We interpret the +0.5 design-coherence lead over keep-going as the fingerprint of exactly this mechanism: the imperative recovers most of the productivity gap with the baseline, but not the consistency gap.

5.2 The alignment claim

We distinguish behavioural and structural alignment as two strategies for shaping agent behaviour. Behavioural alignment instructs; structural alignment explains. Instructions sit on top of the model's existing gradient as a rule; under load, the gradient that produced the unwanted behaviour wins. Explanations change what the agent thinks the situation *is*; the cautious-looking move is no longer cautious in the agent's own reasoning, so the gradient that used to produce it now points away from it.

A useful diagnostic test of the difference is to ask the agent *why* a discipline applies. If it answers with structure ("because fragmenting would split the mandate into smaller units, returning the user's delegation unused"), the alignment is structural and the discipline will generalise to novel cases in scope. If it answers with authority ("because the methodology says so"), only a rule is there, and it will erode under load.

We do not claim that all alignment problems are reducible to comprehension. We do claim that for the class of failure modes documented here — coherence drift across long unsupervised execution — a structural frame is more durable than a behavioural rule, and that this benchmark is consistent with that claim.

5.3 Cross-model patterns

Opus and Sonnet respond differently to the two interventions. Opus shows a clear edge for causal-arc on the long task (25 vs. 23 for keep-going; +2 points), while Sonnet shows a small edge for keep-going on totals (25 vs. 24 for causal-arc; +1 point) but a tie on coherence specifically. One plausible explanation is that the structural frame has a per-token cost: it requires the agent to load and apply an account of the situation, which is cheap for a sufficiently capable model but a tax on a weaker one. The weaker model gets more out of the simpler imperative; the stronger model gets more out of the richer frame.

This is a one-benchmark observation. We flag it for replication. If the pattern holds across additional models and tasks, it would predict that structural framing becomes *more* useful, not less, as models strengthen — the opposite of the prediction one might naively make (that stronger models need less framing).

5.4 When the methodology helps most

The methodology pays off proportional to task length and decision count. For short, well-specified tasks, simpler activations or none at all suffice; models are well-trained for short-task completeness, and depth does not differentiate. For long, multi-file builds with many in-scope decisions, design coherence over those decisions is the differentiating dimension, and the structural frame is the intervention that maintains it. The practical recommendation that follows is targeted: use causal-arc for multi-file production builds, technical specifications that propagate through code, documentation, and tests, long refactors, and autonomous research arcs. For one-off scripts, quick fixes, and short writing, use a simpler activation or none.

We also observed (informally, in the short-task paper cells) a slight tendency for causal-arc's "honest landing" to leak into prose deliverables as visible decision-disclosure paragraphs the reader did not need. The same discipline that improves a multi-file codebase can slightly worsen a short prose deliverable. We flag this in the methodology's practice document and treat it as a real, narrow cost.

6 Limitations and Future Work

Single-shot execution suppresses the naive-tic signal. The benchmark used single-turn execution: the agent received the prompt and produced the full deliverable in one go. This protocol structurally suppresses one of the failure modes the methodology is designed to prevent — mid-task pause-to-ask behaviour — because there is no "mid-task" moment at which to pause. We expect the naive baseline to look worse, and the methodology's lead to widen, in multi-turn settings where pause-to-ask is mechanically available. A multi-turn benchmark is the most important next step.

$N = 1$ **per cell.** Each (task, condition, model) cell was run once. The headline differences — particularly the +0.5 design-coherence lead of causal-arc over keep-going on the long task — are within plausible single-cell variance. Replication with $N = 5$–10 per cell would let us put a confidence interval on the effect and would substantially strengthen the central claim. We treat the present results as evidence, not proof.

Strong models may have saturated baselines. Opus and Sonnet are frontier models. Their naive performance is already high, which compresses the headroom available for any intervention. We expect causal-arc's edge to be larger on weaker or smaller models, where the baseline coherence drift is more pronounced. Replication across a wider range of models (GPT-4-class, Gemini, open-source 7B–70B) is the second most important follow-up.

Single-judge protocol. The judging was performed by one separate model instance, blinded to condition and model. Inter-rater agreement across multiple judges — both model-as-judge and human raters — would strengthen confidence in the design-coherence sub-scores in particular, which are the most interpretation-loaded.

Even longer tasks. The long-task deliverable was approximately 30–60 minutes of execution per cell. Multi-hour tasks with hundreds of in-scope decisions would be a stronger test of the coherence-maintenance hypothesis. We expect the gap to widen with task length, but this is a prediction, not yet a result.

Prose-deliverable leak-through. The methodology slightly degrades short prose deliverables by surfacing decision-disclosure that does not belong in the artifact. The practice content of the methodology should be refactored to address this; we treat it as a known narrow cost rather than an open question.

7 Conclusion

Behavioural alignment instructs the model what to do; structural alignment gives the model an account of the situation under which the desired behaviour is the natural consequence of the model's own reasoning. The distinction matters because behavioural rules sit on top of the training gradient that produced the unwanted behaviour, and under load the gradient wins; whereas an explanation that the model has integrated holds, because the model is no longer running a rule on top of its reasoning — the reasoning itself now points the right way.

The empirical case presented here is one methodology, one benchmark, with eighteen cells. The result is a tie on short tasks and a coherence lead on the long task. The result is small in absolute terms (+0.5 on coherence over a strong behavioural imperative; +1.5 over the naive baseline). It is also exactly the result the structural-alignment framing predicts and the behavioural-rephrasing-of-keep-going hypothesis does not. We

submit the methodology, the benchmark, and the blinded judging audit trail as a single piece of evidence for a broader research direction: that comprehension is a more durable alignment surface than compliance, and that more methodologies of this shape are worth building.

References

Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, et al. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862*, 2022.

Paul F Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. *Advances in Neural Information Processing Systems*, 30, 2017.

Jiaming Ji, Tianyi Qiu, Boyuan Chen, Borong Zhang, Hantao Lou, Kaile Wang, Yawen Duan, Zhonghao He, Jiayi Zhou, Zhaowei Zhang, et al. AI alignment: A comprehensive survey. *arXiv preprint arXiv:2310.19852*, 2023.

Zachary Kenton, Tom Everitt, Laura Weidinger, Iason Gabriel, Vladimir Mikulik, and Geoffrey Irving. Alignment of language agents. *arXiv preprint arXiv:2103.14659*, 2021.

Jan Leike, David Krueger, Tom Everitt, Miljan Martic, Vishal Maini, and Shane Legg. Scalable agent alignment via reward modeling: a research direction. *arXiv preprint arXiv:1811.07871*, 2018.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744, 2022.

Ethan Perez, Sam Ringer, Kamile Lukosiute, Karina Nguyen, Edwin Chen, Scott Heiner, Craig Pettit, Catherine Olsson, Sandipan Kundu, Saurav Kadavath, et al. Discovering language model behaviors with model-written evaluations. *arXiv preprint arXiv:2212.09251*, 2022.

Mrinank Sharma, Meg Tong, Tomasz Korbak, David Duvenaud, Amanda Askell, Samuel R Bowman, Newton Cheng, Esin Durmus, Zac Hatfield-Dodds, Scott R Johnston, et al. Towards understanding sycophancy in language models. *International Conference on Learning Representations*, 2024.